

Introdução à Computação estruturas em C

Alexandre Rademaker

motivação I

Em um aplicativo de contatos, telefones e nomes não são informações separadas, certo?

Um **contato** representa uma entrada no banco de dados e cada entrada terá **campos** que descrevem as informações que temos sobre o referido contato: nome, telefone, endereço, etc.

Durante uma busca e/ou ordenação, em geral, queremos podemos escolher o campo: por nome, por telefone, por empresa, etc.

motivação II

Vamos imaginar que nossos contatos só tivessem nome e telefone.

Poderíamos tentar manter os dados em dois diferentes arrays, mas como garantir a sincronia? Os elementos em posições iguais nos dois arrays correspondem a informações relacionadas ao mesmo contato.

Manter sincroniza em ambos pode ser difícil.

motivação III

phonebook0.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 typedef char* string;
5
6 int main(void) {
7     string names[] = {"Carter", "David"};
8     string numbers[] = {"+1-617-495-1000", "+1-949-468-2750"};
9
10    for (int i = 0; i < 2; i++) {
11        if (strcmp(names[i], "David") == 0) {
12            printf("Found %s\n", numbers[i]);
13            return 0;
14        }
15    }
16    printf("Not found\n");
17    return 1;
18 }
```

estruturas em C I

Mas podemos definir nossas próprias estruturas de dados

phonebook1.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 typedef char* string;
5
6 typedef struct {
7     char name[50];
8     int  idade;
9     char telefone[50];
10 } person;
11
12 int main(void) {
13
14     person a = { "Eduardo", 12, "+85 1234" };
15
16     person b;
17     b.idade = 10;
18
19     // b.name = "..." will not work
20     strcpy(b.name, "Beatriz");
21
22     printf("Person: %s %d\n", b.name, b.idade);
23
24 }
```

structuras em C II

Isto é encapsulamento (encapsulation).

A idéia de que dados relacionados são agrupados, neste caso, nome e número, na mesma estrutura de dados.

A cor de um pixel, com valores de vermelho, verde e azul, também pode ser bem representada por uma estrutura de dados.

Também podemos imaginar que estruturas para armazenar valores racionais ou inteiros grandes. E estruturas podem ter arrays ou outras estruturas como **membros** (campos).

[para saber mais](#)

structuras em C III

```
1 struct pointer {  
2     int x;  
3     int y;  
4 } ;  
5  
6 int main(void) {  
7     struct pointer p; // need the 'struct'  
8     p.x = 10;  
9     p.y = 20;  
10    printf("Point: %d %d\n", p.x, p.y);  
11 }
```

A estrutura foi definida e depois uma variável do tipo foi declarada.

structuras em C IV

```
1 int main(void) {  
2     struct {  
3         int x;  
4         int y;  
5     } p;  
6  
7     p.x = 10;  
8     p.y = 20;  
9     printf("Point: %d %d\n", p.x, p.y);  
10 }
```

A estrutura foi definida e uma variável declarada ao mesmo tipo. Não é possível reusar a definição (não foi nomeada).

structuras em C V

```
1 typedef struct point {
2     int x;
3     int y;
4 } t_point;
5
6 int main(void) {
7     t_point p;
8     p.x = 10;
9     p.y = 20;
10    printf("Point: %d %d\n", p.x, p.y);
11 }
```

O nome `pointer` é opcional, mas necessário para estruturas recursivas (mais tarde!)